Contest Arsenal

PUST_Eternal_Tsukuyomi
Faruk| $Toufique$ | $Mridula$

October 20, 2026

Contents

1 Fast IO **4**

2 CP Math Functions **4**

3 Stress Testing **4**
3.1 brute.cpp 4
3.2 sol.cpp 4
3.3 gen.cpp 4
3.4 run.bat 4

4 STL **4**
4.1 Vector 4
4.2 String 4
4.3 Pair 5
4.4 Set 5
4.5 MultiSet 5
4.6 Map 5
4.7 Deque 5
4.8 Stack 6
4.9 Queue 6
4.10 Priority Queue 6

5 Recursion **6**
5.1 Factorial 6
5.2 Fibonacci 6
5.3 Binary Exponentiation 6
5.4 Coin Change Possible? 6

6 Binary Search **6**
6.1 Binary Search and Integer Bisection . 6
6.2 Binary Search on Answer 6
6.3 Fractional Binary Search 7

7 Sliding Window/ Two Pointers **7**
7.1 Fixed Size Window (Fixed Size Window) 7
7.2 Variable Size Window (Longest Subarray Sum K) 7
7.3 Count DistinctK 7
7.4 Longest Unique Substring 7
7.5 Sliding Window Maximum 7

8 Bit Manipulation **7**
8.1 XOR 7
8.2 Bit function 7
8.3 Bit niye kichu kaj 7

9 Number Theory **7**
9.1 Harmonic Series 7
9.2 Sieve (very very fast) : (O(N log log N)) 7
9.3 Smallest Prime Factor (SPF) : (sieve diye) 8
9.4 Prime factorization : (SPF diye) . . . 8
9.5 Number of common divisors (nod) : harmonic and $_{g}cd()diye$ 8
9.6 BigMOD calculation 8
9.7 Modular Arithmetic 8
9.8 8
9.9 Harmonic Loop 8
9.10 Number of Divisors for all Numbers . 8
9.11 Sum of Divisors for all Numbers . . . 8
9.12 List of Divisors 9
9.13 Sieve of Eratosthenes (O(n log log n)) 9
9.14 Smallest Prime Factor (SPF) 9
9.15 Prime Factorization (sqrt n) 9
9.16 Prime Factorization using SPFO(log n) 9
9.17 Number of Divisors (Single Number) $((\text{p1}^a * p2^b * p3^c)//NOD = (a+1)(b+1)(c+1)$ 9
9.18 Sum of Divisors 9

9.19 Euler Totient Function (Single) (phi(n)=n*(1-1/p1)*(1-1/p2)...) 9
9.20 Euler Totient 1 to N 10
9.21 GCD / LCM 10
9.22 Binary Exponentiation / BigMod (O(log n)) 10
9.23 Modular Inverse (MOD must be prime) 10
9.24 Modular Arithmetic 10
9.25 nCr using Factorial 10
9.26 Distinct Prime Factors Count 10
9.27 Prefix Prime Count 10
9.28 Most Important Number Theory Lines 10

10 Graph **10**
10.1 Adj matrix 10
10.2 Adj list 11
10.3 BFS (Breath First Search) 11
10.4 DFS /depeth /Subtree 11
10.5 Tree Diameter 11
10.6 Cycle Detection + Topological Sort(DFS) 11
10.7 12
10.8 Graph Input and Representation (Unweighted Undirected Graph/ Directed Graph/ Weighted Graph) 12
10.9 BFS (Basic BFS) 12
10.10Shortest Path in Unweighted Graph . 12
10.11Check Graph is Tree or Not 12
10.12Connected Components 12
10.13BFS Path Printing 12
10.14BFS on Grid 13
10.15Multi Source BFS 13
10.16DFS 13
10.17DFS on Tree 13
10.18Depth / Level 13
10.19Subtree Size 13
10.20Tree Diameter (BFS Method) 13
10.21Bipartite Graph 14
10.22Cycle Detection (Undirected) 14
10.23Cycle Detection (Directed) 14
10.24Topological Sort (DFS) 14
10.25Topological Sort (Kahn) 14
10.26DAG Check 14
10.27Graph Complexity 14

11 Basic DP **14**
11.1 Fibonacci DP 14
11.2 Iterative DP 14
11.3 Weight Based Knapsack 14

12 Programming Foundation & Basic Math **15**
12.1 Number System & Bits 15
12.2 Divisors & Primes 15
12.3 GCD, LCM & Factorial 15
12.4 Matrix & Set Theory 15
12.5 Basic Geometry 15
12.6 3D Formulas 15
12.7 Series & Math Basics 15
12.8 Permutation, Combination & Probability 15

13 Sort **15**
13.1 Custom Sort 15

14 Default Template **15**

15 Data Structures **16**
15.1 Binary Indexed Tree 16
15.2 Disjoint Set Union 16
15.3 Segment Tree (Basic Max) 16
15.4 Segment Tree with Lazy 16
15.5 Segment Tree (AP Lazy) 17
15.6 Sparse Table (RMQ) 17
15.7 Monotonic Queue 17
15.8 Trie Tree 17
15.9 Merge Sort Tree 17
15.10Mo's Algorithm 18
15.11SQRT Decomposition 18
15.12Ordered Set (Pbds) 18

16 Graph Theory **18**
16.1 Bipartite Matching (Hopcroft-Karp) . 18
16.2 SCC (Kosaraju) 19
16.3 MST (Kruskal) 19
16.4 Topological Sort + Cycle 19
16.5 Articulation Bridge 19
16.6 Articulation Point 19
16.7 LCA (Binary Lifting) 20
16.8 Centroid Decomposition 20
16.9 Max Flow (Ford-Fulkerson) 20
16.10Min Cost Max Flow 21
16.11Bellman-Ford / Neg. Cycle 21
16.12DSU on Tree 21

17 Number Theory **22**
17.1 Extended Euclid + Mod Inverse . . . 22
17.2 BigMod, Factorial, nCr 22
17.3 Euler Totient (Sieve) 22
17.4 Sieve of Eratosthenes 22
17.5 Miller-Rabin Primality 22
17.6 Discrete Logarithm 22
17.7 Sum / Number of Divisors 23

18 String **23**
18.1 KMP 23
18.2 Z-Algorithm 23
18.3 Hashing (Double Hash) 23
18.4 Suffix Array 24
18.5 Palindromic Tree (Eertree) 24
18.6 Manacher's Algorithm 24
18.7 String Matching (Bitset) 24

19 Math & Matrix **25**

19.1 Matrix Struct 25

20 Geometry **25**

20.1 Geometry Formulas 25

21 Important Formulas **25**

21.1 Useful Identities 25

21.2 Bit Macros 25

1 Fast IO

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

2 CP Math Functions

```
sqrt(n)   // square root
pow(a,b)  // a^b
abs(x)    // absolute value
ceil(x)   // upper integer
floor(x)  // lower integer
```

3 Stress Testing

3.1 brute.cpp

```
int main(){

}
```

3.2 sol.cpp

```
int main(){

}
```

3.3 gen.cpp

```
int main(){

}
```

3.4 run.bat

```
@echo off
gen >in
sol <in >out
brute <in >ok
fc out ok
if ErrorLevel 1 exit /b
run
```

4 STL

4.1 Vector

```
1.Declaration
  vector<int> v;
  vector<int> v(n);
  vector<int> v(n,0);
2.Input
  for(auto &x:v)
    cin >> x;
3.Output
  for(auto x:v)
    cout << x << ' ';
4.push_back / pop_back
  v.push_back(x);
  v.pop_back();
5.Size / Empty
  v.size()
  v.empty()
6.Clear / Resize / Assign
  v.clear();
  v.resize(n);
  v.assign(n,5);
7.Front / Back
  v.front()
  v.back()
8.Sort
  sort(v.begin(),v.end());
  sort(v.rbegin(),v.rend());
9.Reverse
  reverse(v.begin(),v.end());
10.Unique
   sort(v.begin(),v.end());
   v.erase( unique( v.begin(),v.end()),v.end
      ↪() );
11.Max / Min Element
   *max_element(v.begin(),v.end());
   *min_element(v.begin(),v.end());
12.Sum
   accumulate(v.begin(),v.end(),0LL);
13.Count
   count(v.begin(),v.end(),x);
14.lower_bound / upper_bound
   lower_bound(v.begin(),v.end(),x)-v.begin
      ↪();
   upper_bound(v.begin(),v.end(),x)-v.begin
      ↪();
14.Rotate
   rotate(v.begin(), v.begin()+k, v.end());
      ↪//bame k ghor rotate
15.Next Permutation
   next_permutation(v.begin(), v.end());
16.Previous Permutation
   prev_permutation(v.begin(), v.end());
```

4.2 String

```
1.Declaration
  string s;
2.Input
  cin >> s;
  getline(cin,s);
3.Length
  s.size()
  s.length()
4.Traversal
  for(char c:s)
    cout << c;
5.Concatenation
  string t=s+"abc";
6.Reverse
  reverse(s.begin(),s.end());
7.Sort
  sort(s.begin(),s.end());
8.Upper / Lower
  toupper(c)
  tolower(c)
9.Erase Part
  s.erase(pos,len);
10.Substring
   string t = s.substr(l, len);
11.Find
   s.find("abc");
12.Count Character
   count(s.begin(),s.end(),'a');
13.to_string
   to_string(x);
14.String to Number
   int x = stoi(s);
   long long y = stoll(s);
   double z = stod(s);
15.next_permutation
   //sob permutation generate korte:
   sort(s.begin(),s.end());
   do{
     cout << s << '\n';
   }
```

```
  while(next_permutation(s.begin(),s.end())
     ↪);
16.prev_permutation
  prev_permutation(s.begin(),s.end());
17.All Substrings
  for(int i=0;i<n;i++){
      for(int j=1;j<=n-i;j++){
         cout<<s.substr(i,j)<<'\n';
      }
   }
19.Remove Duplicate Characters
  sort(s.begin(),s.end());
  s.erase(unique(s.begin(),s.end()),s.end()
     ↪);
20.Substring Exists?
  if(s.find("abc") != string::npos)
21.Insert
  s.insert(pos, "abc");
22.Replace
 s.replace(pos, len, "xyz");
23.All Lowercase
  all_of(s.begin(), s.end(), ::islower);
24.All Uppercase
  all_of(s.begin(), s.end(), ::isupper);
25.All Digits
  all_of(s.begin(), s.end(), ::isdigit);
26.Split String (stringstream)
  string s="10 20 30";
  stringstream ss(s);
  int x;
  while(ss>>x)
    cout<<x<<' ';
```

4.3 Pair

```
1.Declaration
 pair<int,int> p;
2.Input
 cin >> p.first >> p.second;
3.make_pair
 make_pair(a,b);
4.Nested Pair
 pair<int,pair<int,int>> p;
5.Vector of Pair
 vector<pair<int,int>> v;
6.Sort Pair
 sort(v.begin(),v.end());
```

4.4 Set

```
1.Declaration
 set<int> st;
2.Insert/Erase
 st.insert(x);
 st.erase(x);
 auto it = st.find(5);
 st.erase(it);
3.Find /Count
 st.find(x);
 st.count(x);
4.lower_bound/ upper_bound
 st.lower_bound(x);
 st.upper_bound(x);
5.Previous Element
 auto it=st.lower_bound(x);
  if(it!=st.begin()){
    --it;
    cout << *it;
  }
6.Remove Smallest/Remove Largest
 st.erase(st.begin());
 auto it=prev(st.end());
 st.erase(it);
```

4.5 MultiSet

```
1.Declaration
 multiset<int> ms;
2.Insert
 ms.insert(x);
3.Erase One Occurrence
 ms.erase(ms.find(x));
4.Count
 ms.count(x);
5.Frequency using Bounds
 distance(ms.lower_bound(x),ms.upper_bound
    ↪(x));
6.Last Element Erase
 auto it = prev(ms.end());
 ms.erase(it);
7.Frequency via Range
 int freq =distance( ms.lower_bound(x), ms
    ↪.upper_bound(x) );
8.Merge (C++17)
 multiset<int> a,b;
 a.merge(b);
4.
 insert()
 find()
 count()
 erase(iterator)
 lower_bound()
 upper_bound()
 begin()
 rbegin()
```

4.6 Map

```
1.Declaration
 map<int,int> mp;
2.Insert
 mp[x]=5;
3.Frequency
 mp[x]++;
4.Traverse
 for(auto [x,y]:mp){
   cout << x<< ' '<< y<< '\n';
 }
5.Find
 mp.find(x);
6.Erase
 mp.erase(x);
7.Count
 mp.count(x)
8.Custom Order
 map<int,int,greater<int>> mp;
9.Erase by Iterator
 auto it = mp.find(x);
 if(it != mp.end()) mp.erase(it);
```

4.7 Deque

```
1.Declaration
 deque<int> dq;
2.Functions
 dq.push_front(x);
 dq.push_back(x);
 dq.pop_front();
```

```
  dq.pop_back();
  dq.front();
  dq.back();
  size()
  empty()
3.advanced level-a :
  0-1 BFS
  Monotonic Queue
  Sliding Window Maximum
```

4.8 Stack

```
1.Declaration
  stack<int> st;
2.Functions
  st.push(x);
  st.pop();
  st.top();
  st.empty();
  st.size();
3.Valid Bracket
  stack<char> st;
  for(char c:s){
    if(c=='(') st.push(c);
    else{
      if(st.empty()){
        cout<<"NO\n";
        return;
      }
      st.pop();
    }
  }
  cout <<(st.empty()?"YES":"NO");
```

4.9 Queue

```
1.Declaration
  queue<int> q;
2.Functions
  q.push(x);
  q.pop();
  q.front();
  q.empty();
```

4.10 Priority Queue

```
1.Max Heap
  priority_queue<int> pq;
2.Min Heap
  priority_queue<int,vector<int>,greater<
     ↪int>> pq;
3.Functions
  pq.push(x);
  pq.pop();
  pq.top();
  pq.empty();
```

5 Recursion

5.1 Factorial

```
long long fact(long long n){
   if(n==0)
      return 1;
   return n*fact(n-1);
}
```

5.2 Fibonacci

```
long long fib(long long n){
   if(n<=1)
      return n;
   return fib(n-1)+fib(n-2);
}
```

5.3 Binary Exponentiation

```
long long power(
   long long a,
   long long b
){
   if(b==0)
      return 1;
   long long x=power(a,b/2);
   x=x*x;
   if(b&1)
      x*=a;
   return x;
}
```

5.4 Coin Change Possible?

```
bool possible(
   int idx,
   int sum
){
   if(sum==target)
      return true;
   if(idx==n || sum>target)
      return false;
   return possible(idx+1,sum+a[idx]) ||
      ↪possible( idx+1,sum);
}
```

6 Binary Search

6.1 Binary Search and Integer Bisection

```
bool ok(int m,int k, vll v){
for(int i=0;i<v.size();i++){
      if(v[i]<m) k-=(m-v[i]);
   }
   return k>=0;
}
int l=0,r=1e10,ans=0;
while(l<=r){
      int m=(l+r)/2;
      if(ok(m,k,v)){
         ans=m;
         l=m+1;
      }else{
         r=m-1;
      }
   }
```

6.2 Binary Search on Answer

```
bool ok(int mid){
}
//----------
int l=0;
int r=1e18;
int ans=0;
while(l<=r){
   int mid=(l+r)/2;
   if(ok(mid)){
      ans=mid;
      l=mid+1;
   }
   else{
      r=mid-1;
   }
}
```

6.3 Fractional Binary Search

```
bool ok(double mid){
   //condiation
}
//-------------
double l=0;
double r=1e9;
for(int i=0;i<100;i++){
   double mid=(l+r)/2;
   if(ok(mid))
      l=mid;
   else
      r=mid;
}
```

7 Sliding Window/ Two Pointers

7.1 Fixed Size Window (Fixed Size Window)

```
int sum=0;
for(int i=0;i<k;i++)
   sum+=a[i];
int ans=sum;
for(int i=k;i<n;i++){
   sum+=a[i];
   sum-=a[i-k];
   ans=max(ans,sum);
}
```

7.2 Variable Size Window (Longest Subarray Sum K)

```
int l=0,sum=0,ans=0;
for(int r=0;r<n;r++){
   sum+=a[r];
   while(sum>k){
      sum-=a[l];
      l++;
   }
   ans=max(ans,r-l+1);
}
```

7.3 Count DistinctK

```
map<int,int> freq;
int l=0;
for(int r=0;r<n;r++){
   freq[a[r]]++;
   while(freq.size()>k){
      freq[a[l]]--;
      if(freq[a[l]]==0)
         freq.erase(a[l]);
      l++;
   }
}
```

7.4 Longest Unique Substring

```
map<char,int> freq;
int l=0,ans=0;
for(int r=0;r<s.size();r++){
   freq[s[r]]++;
   while(freq[s[r]]>1){
      freq[s[l]]--;
      l++;
   }
   ans=max(ans,r-l+1);
}
```

7.5 Sliding Window Maximum

```
deque<int> dq;
for(int i=0;i<n;i++){
   while(!dq.empty() &&
        a[dq.back()] <= a[i])
      dq.pop_back();
   dq.push_back(i);
   while(!dq.empty() &&
        dq.front() <= i-k)
      dq.pop_front();
   if(i >= k-1)
      cout << a[dq.front()] << " ";
}
```

8 Bit Manipulation

8.1 XOR

```
a^b=c;
a^c=b;
```

8.2 Bit function

```
1. __biltin_popcount(x) //ato gula 1 ache
2. __lg(x) //mostsignificat bin number
```

8.3 Bit niye kichu kaj

```
1. Left shift ( << ) man bare
2. Right shift ( >> )
3. Check Kth bit on or off
  ==(n>>k)&1
4 Turn on kth bit
  ==(n|(1<<k))
5.Turn off kth bit
  ==(n&(~(1<<k)))
6.Toggle kth bit
 ==(n^(1<<k))
7. 2 ar power calculate
 == 1<<k
8. swap(a,b)
  a=a^b;
  b=a^b;
  a=a^b;
```

9 Number Theory

9.1 Harmonic Series

```
const int N=1e5+5;
void harmonic(){
   for(int i=1;i<N;i++){
      for(int j=i;j<N;j+=i){
         nod[j]++;
      }
   }
}
```

9.2 Sieve (very very fast) : (O(N log log N))

```
const int N=1e7;
int cnt=0;
vector<bool> isprime(N,true);
vll prime;
void sieve(){
   for(int i=3;i*i<N;i+=2){
      if(isprime[i]){
         for(int j=i*i;j<N;j+=(i+i)){
            isprime[j]=false;
```

```
            cnt++;
         }
      }
   }
   prime.push_back(2);
   for(int i=3;i<N;i+=2){
      if(isprime[i])prime.push_back(i);
   }
}
void solve(){
   cout<<cnt<<nl;
   frg(x,prime)cout<<x<<' ';
}
```

9.3 Smallest Prime Factor (SPF) : (sieve diye)

```
const int N=1e6+5;
vector<bool>isprime(N,true);
vll SPF(N);
void sieve(){
   for(int i=0;i<N;i++) SPF[i]=i;
   for(int i=2 ;i*i<N;i++){
      if(isprime[i]){
         for(int j=i*i;j<N;j+=i){
            isprime[j]=false;
            if(SPF[j]==j)SPF[j]=i;
         }
      }
   }
}
int main(){
   spf();
   int x;cin>>x;
   cout<<SPF[x]<<nl;
}
```

9.4 Prime factorization : (SPF diye)

```
void prime_factorization(){
   int n;cin>>n;
   while(n>1){
      int x=n;
      int e=0;
      while(n%SPF[x]==0){
         e++;
         n/=SPF[x];
      }
      cout<<SPF[x]<<' '<<e<<nl; // P
   }
}
```

9.5 Number of common divisors (nod) : harmonic and $_{g}cd()diye$

```
vll nod[N];
void harmonic(){
   for(int i=1;i<N;i++){
      for(int j=i;j<N;j+=i){
         nod[j].push_back(i);
      }
   }
}
int main(){
   harmonic();
   int a,b,c;cin>>a>>b>>c;
   int n=__gcd(a,b);
   n=__gcd(n,c);
   for(auto x:nod[n])cout<<x<<' ';
}
```

9.6 BigMOD calculation

```
const int mod=1e9+7;
int bigmod(int a,int b){
   if(b==0)return 1;
   if(b%2==1){
      return (a%mod*bigmod(a,b-1)%mod)%mod;
   }
   int tmp =bigmod(a,b/2)%mod;
   return (tmp*tmp)%mod;
}
```

9.7 Modular Arithmetic

```
1.a%mod if(a<0)
  (((a%mod)+mod)%mod)
2.Addition ( a+b % mod )
  (a+b)%MOD
  ((a%MOD)+(b%MOD))%MOD
3.Subtraction ( a-b % mod )
  (a-b+MOD)%MOD
4.Multiplication ( a*b % mod )
  (a%MOD)*(b%MOD)%MOD
5.BigMod ( a^b % mod )
  const int mod=1e9+7;
  int bigmod(int a,int b){
    if(b==0)return 1;
    if(b%2==1){
       return (a%mod*bigmod(a,b-1)%mod)%mod;
    }
    int tmp =bigmod(a,b/2)%mod;
    return (tmp*tmp)%mod;
}
6. Modular Division ( a/b % mod )
```

9.8

9.9 Harmonic Loop

```
// O(n log n)
for(int i=1;i<=n;i++){
   for(int j=i;j<=n;j+=i){
      // i divides j
   }
}
```

9.10 Number of Divisors for all Numbers

```
const int N = 1e6+5;
int nod[N];
void harmonic(){
   for(int i=1;i<N;i++){
      for(int j=i;j<N;j+=i){
         nod[j]++;
      }
   }
}
```

9.11 Sum of Divisors for all Numbers

```
const int N = 1e6+5;
long long sod[N];
void harmonic(){
   for(int i=1;i<N;i++){
      for(int j=i;j<N;j+=i){
         sod[j]+=i;
      }
   }
}
```

9.12 List of Divisors

```
const int N = 1e5+5;
vector<int> divs[N];
void harmonic(){
   for(int i=1;i<N;i++){
      for(int j=i;j<N;j+=i){
         divs[j].push_back(i);
      }
   }
}
```

9.13 Sieve of Eratosthenes (O(n log log n))

```
const int N = 1e7+5;
vector<bool> isPrime(N,true);
vector<int> prime;
void sieve(){
   isPrime[0]=isPrime[1]=false;
   for(int i=4;i<N;i+=2)
      isPrime[i]=false;
   for(int i=3;i*i<N;i+=2){
      if(isPrime[i]){
         for(int j=i*i;j<N;j+=i+i){
            isPrime[j]=false;
         }
      }
   }
   prime.push_back(2);
   for(int i=3;i<N;i+=2){
      if(isPrime[i])
         prime.push_back(i);
   }
}
```

9.14 Smallest Prime Factor (SPF)

```
const int N = 1e6+5;
int spf[N];
void sieve(){
   for(int i=1;i<N;i++)
      spf[i]=i;
   for(int i=2;i*i<N;i++){
      if(spf[i]==i){
         for(int j=i*i;j<N;j+=i){
            if(spf[j]==j)
               spf[j]=i;
         }
      }
   }
}
```

9.15 Prime Factorization (sqrt n)

```
vector<pair<long long,int>> primeFactor(
   ↪long long n){
   vector<pair<long long,int>> res;
   for(long long i=2;i*i<=n;i++){
      if(n%i==0){
         int cnt=0;
         while(n%i==0){
            cnt++;
            n/=i;
         }
         res.push_back({i,cnt});
      }
   }
   if(n>1)
      res.push_back({n,1});
   return res;
}
```

9.16 Prime Factorization using SPFO(log n)

```
void factorize(int n){
   while(n>1){
      int p=spf[n];
      int cnt=0;
      while(n%p==0){
         cnt++;
         n/=p;
      }
      cout << p << " " << cnt << '\n';
   }
}
```

9.17 Number of Divisors (Single Number) ((p1^a * p2^b * p3^c)//$NOD = (a+1)(b+1)(c+1)$

```
long long nod(long long n){
   long long ans=1;
   for(long long i=2;i*i<=n;i++){
      if(n%i==0){
         int cnt=0;
         while(n%i==0){
            cnt++;
            n/=i;
         }
         ans*=1LL*(cnt+1);
      }
   }
   if(n>1)
      ans*=2;
   return ans;
}
```

9.18 Sum of Divisors

```
long long sod(long long n){
   long long ans=1;
   for(long long i=2;i*i<=n;i++){
      if(n%i==0){
         long long sum=1;
         long long p=1;
         while(n%i==0){
            n/=i;
            p*=i;
            sum+=p;
         }
         ans*=sum;
      }
   }
   if(n>1)
      ans*=(1+n);
   return ans;
}
```

9.19 Euler Totient Function (Single) (phi(n)=n*(1-1/p1)*(1-1/p2)...)

```
long long phi(long long n){
   long long ans=n;
   for(long long i=2;i*i<=n;i++){
      if(n%i==0){
         while(n%i==0)
```

```
            n/=i;
         ans-=ans/i;
      }
   }
   if(n>1)
      ans-=ans/n;
   return ans;
}
```

9.20 Euler Totient 1 to N

```
const int N = 1e6+5;
int phi[N];
void phi_1_to_n(){
   for(int i=0;i<N;i++)
      phi[i]=i;
   for(int i=2;i<N;i++){
      if(phi[i]==i){
         for(int j=i;j<N;j+=i){
            phi[j]-=phi[j]/i;
         }
      }
   }
}
```

9.21 GCD / LCM

```
long long g = __gcd(a,b);
long long l =
(a/__gcd(a,b))*b;
```

9.22 Binary Exponentiation / BigMod (O(log n))

```
const long long MOD = 1e9+7;
long long bigmod(
   long long a,
   long long b
){
   long long res=1;
   while(b){
      if(b&1)
         res=res*a%MOD;
      a=a*a%MOD;
      b>>=1;
   }
   return res;
}
```

9.23 Modular Inverse (MOD must be prime)

```
long long modInverse(long long a){
   return bigmod(a,MOD-2);
}
```

9.24 Modular Arithmetic

```
1.Addition
  (a+b)%MOD
2.Subtraction
  (a-b+MOD)%MOD
3.Multiplication
  (a%MOD)*(b%MOD)%MOD
4.Division
  a* modInverse(b) % MOD
```

9.25 nCr using Factorial

```
const int N = 1e6+5;
long long fact[N];
long long invFact[N];
void factorial(){
   fact[0]=1;
   for(int i=1;i<N;i++)
      fact[i]=fact[i-1]*i%MOD;
}
1. Inverse Factorial
void inverseFactorial(){
   invFact[N-1]=
   bigmod(fact[N-1],MOD-2);
   for(int i=N-2;i>=0;i--){
      invFact[i]=
      invFact[i+1]*(i+1)%MOD;
   }
}
2.nCr
long long nCr(int n,int r){
   if(r<0 || r>n)
      return 0;
   return fact[n]*
        invFact[r]%MOD*
        invFact[n-r]%MOD;
}
```

9.26 Distinct Prime Factors Count

```
const int N = 1e6+5;
int cnt[N];
void sieve(){
   for(int i=2;i<N;i++){
      if(cnt[i]==0){
         for(int j=i;j<N;j+=i){
            cnt[j]++;
         }
      }
   }
}
```

9.27 Prefix Prime Count

```
const int N = 1e6+5;
int pre[N];
void build(){
   for(int i=1;i<N;i++){
      pre[i]=pre[i-1]+isPrime[i];
   }
}
//Query:----
prime count in [l,r]=pre[r]-pre[l-1]
```

9.28 Most Important Number Theory Lines

```
__gcd(a,b)
bigmod(a,b)
bigmod(a,MOD-2)
(a-b+MOD)%MOD
for(int j=i;j<=n;j+=i)
spf[n]
phi(n)
nod(n)
sod(n)
```

10 Graph

10.1 Adj matrix

```
void solve(){
   int n,m;cin>>n>>m;
   int adj[n+1][n+1];
   for(int i=0;i<n+1;i++){
      for(int j=0;j<n+1;j++){
```

```
            adj[i][j]=0;
        }
    }
    for(int i=0;i<m;i++){
        int u,v;cin>>u>>v;
        adj[u][v]=1;
        adj[v][u]=1;
    }
    for(int i=1;i<n+1;i++){
        for(int j=1;j<n+1;j++){
            cout<<adj[i][j]<<' ';
        }
        cout<<nl;
    }
}
```

10.2 Adj list

```
void solve(){
    int n,m;cin>>n>>m;
    vector<int> adj[n+1];
    for(int i=0;i<m;i++){
        int u,v;cin>>u>>v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    for(int i=1;i<n+1;i++){
        cout<<i<<" --> {";
        for(auto j:adj[i]){
            cout<<j<<' ';
        }
        cout<<"}\n";
    }
}
```

10.3 BFS (Breath First Search)

```
void solve(){
    int n,m;cin>>n>>m;
    vector<int> adj[n+1];
    for(int i=0;i<m;i++){
        int u,v;cin>>u>>v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    int visited[n+1];
    int level[n+1];
    for(int i=0;i<n+1;i++) visited[i]=0;
    queue<int>q;
    int sorce;sorce=1;
    q.push(sorce);
    visited[sorce]=1;
    level[sorce]=1;
    while(!q.empty()){
        int node=q.front();
        cout<<node<<' ';
        for(auto u:adj[node]){
            if(visited[u])continue;
            visited[u]=1;
            level[u]=level[node]+1;
            q.push(u);
        }
        q.pop();
    }
}
```

10.4 DFS /depeth /Subtree

```
const int N=2e5+5;
vector<int>adj[N];
int visited[N];
int depth[N];
int subtree_sz[N];
void dfs(int node){
    visited[node]=1;
    subtree_sz[node]=1;
    for(auto u:adj[node]){
        if(visited[u]==0){
            depth[u]=depth[node]+1;
            dfs(u);
            subtree_sz[node]+=subtree_sz[u];
        }
    }
}
void solve(){
    int n,m;cin>>n>>m;
    for(int i=0;i<m;i++){
        int u,v;cin>>u>>v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    dfs(1);
    cout<<"Subtree \n";
    for(int i=1;i<=n;i++){
        cout<<i<<" ---> "<<subtree_sz[i]<<nl;
    }
    cout<<nl<<"Depth: \n";
    for(int i=1;i<=n;i++){
        cout<<i<<" --->> "<<depth[i]<<nl;
    }
}
```

10.5 Tree Diameter

```
const int N=2e5+5;
vector<int> adj[N];
int forthest_node,forthest_dist;
void fun(int node,int parent,int dist){
    if(dist>forthest_dist){
        forthest_node=node;
        forthest_dist=dist;
    }
    for(auto u:adj[node]){
        if(u==parent)continue;
        fun(u,node,dist+1);
    }
}
void solve(){
    int n;cin>>n;
    for(int i=0;i<n-1;i++){
        int u,v;cin>>u>>v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    forthest_dist=0;
    forthest_node=1;
    fun(1,-1,0) ;
    fun(forthest_node,-1,0);
    cout<<forthest_dist<<nl;
}
```

10.6 Cycle Detection + Topological Sort(DFS)

```
const int N=2e5+5;
vector<int>adj[N];
int mark[N];
bool cycle=0;
```

```
vector<int >topsort;
void dfs(int node){
   if(mark[node]==2)return;
   if(mark[node]==1){
      cycle=1;
      return;
   }
   mark[node]=1;
   for(auto u:adj[node])dfs(u);
   mark[node]=2;
   topsort.push_back(node);
}
void solve(){
   int n,m;cin>>n>>m;
   for(int i=0;i<m;i++){
      int u,v;cin>>u>>v;
      adj[u].push_back(v);
   }
   for(int i=1;i<=n;i++){
      if(!mark[i])dfs(i);
   }
   if(cycle){
      cout<<"Cycle Found\n";
   }
   reverse(topsort.begin(),topsort.end());
   for(auto u:topsort)cout<<u<<' ';
   cout<<'\n';
}
```

10.7

10.8 Graph Input and Representation (Unweighted Undirected Graph/ Directed Graph/ Weighted Graph)

```
1.Unweighted Undirected Graph
const int N = 2e5+5;
vector<int> adj[N];
int main(){
   int n,m;
   cin >> n >> m;
   for(int i=0;i<m;i++){
      int u,v;
      cin >> u >> v;
      adj[u].push_back(v);
      adj[v].push_back(u);
   }
}
2.Directed Graph
adj[u].push_back(v);
3.Weighted Graph
vector<pair<int,int>> adj[N];
adj[u].push_back({v,w});
adj[v].push_back({u,w});
```

10.9 BFS (Basic BFS)

```
vector<int> vis(N);
void bfs(int src){
   queue<int> q;
   q.push(src);
   vis[src]=1;
   while(!q.empty()){
      int u=q.front();
      q.pop();
      for(auto v:adj[u]){
         if(!vis[v]){
            vis[v]=1;
            q.push(v);
         }
      }
   }
}
```

10.10 Shortest Path in Unweighted Graph

```
vector<int> dist(N,-1);
void bfs(int src){
   queue<int> q;
   q.push(src);
   dist[src]=0;
   while(!q.empty()){
      int u=q.front();
      q.pop();
      for(auto v:adj[u]){
         if(dist[v]==-1){
            dist[v]=dist[u]+1;
            q.push(v);
         }
      }
   }
}
```

10.11 Check Graph is Tree or Not

```
//connected + edges = n-1
int cnt=0;
void bfs(int src){
   queue<int> q;
   q.push(src);
   vis[src]=1;
   while(!q.empty()){
      int u=q.front();
      q.pop();
      cnt++;
      for(auto v:adj[u]){
         if(!vis[v]){
            vis[v]=1;
            q.push(v);
         }
      }
   }
}
//------------------
if(cnt==n && m==n-1)
   cout<<"Tree\n";
```

10.12 Connected Components

```
int cc=0;
for(int i=1;i<=n;i++){
   if(!vis[i]){
      cc++;
      bfs(i);
   }
}
```

10.13 BFS Path Printing

```
vector<int> par(N,-1);
void bfs(int src){
   queue<int> q;
   q.push(src);
   vis[src]=1;
   while(!q.empty()){
      int u=q.front();
      q.pop();
      for(auto v:adj[u]){
```

```
            if(!vis[v]){
               vis[v]=1;
               par[v]=u;
               q.push(v);
            }
         }
      }
}
//-----------
//Path Restore
//----------
vector<int> path;
int node=dest;
while(node!=-1){
   path.push_back(node);
   node=par[node];
}
reverse(path.begin(),path.end());
```

10.14 BFS on Grid

```
int dx[]={-1,1,0,0};
int dy[]={0,0,-1,1};
bool valid(int x,int y){
   return x>=0 && y>=0 && x<n && y<m;
}
//---------------------
queue<pair<int,int>> q;
q.push({sx,sy});
vis[sx][sy]=1;
while(!q.empty()){
   auto [x,y]=q.front();
   q.pop();
   for(int i=0;i<4;i++){
 i= 1-->U 2-->D 3-->L 4-->R
      int nx=x+dx[i];
      int ny=y+dy[i];
      if(valid(nx,ny) && !vis[nx][ny]){
         vis[nx][ny]=1
         q.push({nx,ny});
      }
   }
}
```

10.15 Multi Source BFS

```
queue<int> q;
for(int i=1;i<=n;i++){
   if(source[i]){
      q.push(i);
      dist[i]=0;
   }
}
//---------------
while(!q.empty()){
   int u=q.front();
   q.pop();
   for(auto v:adj[u]){
      if(dist[v]==-1){
         dist[v]=dist[u]+1;
         q.push(v);
      }
   }
}
```

10.16 DFS

```
vector<int> vis(N);
void dfs(int u){
   vis[u]=1;
   for(auto v:adj[u]){
      if(!vis[v])
         dfs(v);
   }
}
```

10.17 DFS on Tree

```
void dfs(int u,int p){
   for(auto v:adj[u]){
      if(v==p)
         continue;
      dfs(v,u);
   }
}
```

10.18 Depth / Level

```
int depth[N];
void dfs(int u,int p){
   for(auto v:adj[u]){
      if(v==p)
         continue;
      depth[v]=depth[u]+1;
      dfs(v,u);
   }
}
```

10.19 Subtree Size

```
int sub[N];
void dfs(int u,int p){
   sub[u]=1;
   for(auto v:adj[u]){
      if(v==p)
         continue;
      dfs(v,u);
      sub[u]+=sub[v];
   }
}
```

10.20 Tree Diameter (BFS Method)

```
pair<int,int> bfs(int src){
   vector<int> dist(N,-1);
   queue<int> q;
   q.push(src);
   dist[src]=0;
   while(!q.empty()){
      int u=q.front();
      q.pop();
      for(auto v:adj[u]){
         if(dist[v]==-1){
            dist[v]=dist[u]+1;
            q.push(v);
         }
      }
   }
   int mx=0,node=src;
   for(int i=1;i<=n;i++){
      if(dist[i]>mx){
         mx=dist[i];
         node=i;
      }
   }
   return {node,mx};
}
\\------------
auto [u,d1]=bfs(1);
auto [v,d2]=bfs(u);
cout<<d2<<'\n';
```

10.21 Bipartite Graph

```
vector<int> color(N,-1);
bool bfs(int src){
   queue<int> q;
   q.push(src);
   color[src]=0;
   while(!q.empty()){
      int u=q.front();
      q.pop();
      for(auto v:adj[u]){
         if(color[v]==-1){
            color[v]=color[u]^1;
            q.push(v);
         }
         else if(color[v]==color[u])
            return false;
      }
   }
   return true;
}
```

10.22 Cycle Detection (Undirected)

```
bool dfs(int u,int p){
   vis[u]=1;
   for(auto v:adj[u]){
      if(!vis[v]){
         if(dfs(v,u))
            return true;
      }
      else if(v!=p)
         return true;
   }
   return false;
}
```

10.23 Cycle Detection (Directed)

```
vector<int> vis(N);
bool cycle=false;
void dfs(int u){
   vis[u]=1;
   for(auto v:adj[u]){
      if(vis[v]==0)
         dfs(v);
      else if(vis[v]==1)
         cycle=true;
   }
   vis[u]=2;
}
//---------------
0 = unvisited
1 = processing
2 = processed
```

10.24 Topological Sort (DFS)

```
vector<int> topo;
void dfs(int u){
   vis[u]=1;
   for(auto v:adj[u]){
      if(!vis[v])
         dfs(v);
   }
   topo.push_back(u);
}
//-----------
reverse(topo.begin(),topo.end());
```

10.25 Topological Sort (Kahn)

```
queue<int> q;
for(int i=1;i<=n;i++){
   if(indeg[i]==0)
      q.push(i);
}
//------------
while(!q.empty()){
   int u=q.front();
   q.pop();
   topo.push_back(u);
   for(auto v:adj[u]){
      indeg[v]--;
      if(indeg[v]==0)
         q.push(v);
   }
}
```

10.26 DAG Check

```
if(topo.size()==n)
   cout<<"DAG\n";
else
```

10.27 Graph Complexity

```
BFS                  O(V+E)
DFS                  O(V+E)
Connected Components O(V+E)
Topological Sort     O(V+E)
Dijkstra             O(E log V)
```

11 Basic DP

11.1 Fibonacci DP

```
vector<long long> dp(N,-1);
long long fib(int n){
   if(n<=1)
      return n;
   if(dp[n]!=-1)
      return dp[n];
   return dp[n]=
   fib(n-1)+fib(n-2);
}
```

11.2 Iterative DP

```
dp[0]=0;
dp[1]=1;
for(int i=2;i<=n;i++){
   dp[i]=dp[i-1]+dp[i-2];
}
```

11.3 Weight Based Knapsack

```
void weight_based_knapsack(){
   int n,W;
   cin >> n >> W;
   vector<pair<int,int>> a(n);
   for(auto &[wt,val]:a)
      cin >> wt >> val;
   vector<int> dp(W+1,-1);
   dp[0]=0;
   for(auto &[wt,val]:a){
      for(int w=W;w>=0;w--){
         if(dp[w]==-1) continue;
         if(w+wt>W) continue;
         dp[w+wt]=max(dp[w+wt],dp[w]+val);
      }
   }
   cout <<*max_element(dp.begin(),dp.end())
      ↪<< nl;
}
```

12 Programming Foundation & Basic Math

12.1 Number System & Bits

$$\text{Even} = 2k \quad (\text{where } k \in Z)$$

$$\text{Odd} = 2k+1 \quad (\text{where } k \in Z)$$

$$(a_n a_{n-1} \dots a_1 a_0)_r = (a_n \times r^n) + \dots + (a_0 \times r^0)$$

12.2 Divisors & Primes

$$N = p_1^{a_1} \times p_2^{a_2} \times \dots \times p_k^{a_k}$$

$$d(N) = (a_1+1)(a_2+1)\cdots(a_k+1)$$

$$\text{Loop Limit} \le \sqrt{N}, \quad \text{Digit} = N \bmod 10$$

12.3 GCD, LCM & Factorial

$$\gcd(a,b) = \gcd(b,\ a \bmod b)$$

$$\text{lcm}(a,b) = \frac{|a \times b|}{\gcd(a,b)}$$

$$n! = n \times (n-1) \times \dots \times 1$$

$$E_5(N!) = \left\lfloor \frac{N}{5} \right\rfloor + \left\lfloor \frac{N}{25} \right\rfloor + \left\lfloor \frac{N}{125} \right\rfloor + \dots$$

12.4 Matrix & Set Theory

$$C_{ij} = A_{ij} + B_{ij}, \qquad C_{ij} = \sum_{k=1}^{M} A_{ik} \times B_{kj}$$

$$n(A \cup B) = n(A) + n(B) - n(A \cap B)$$

12.5 Basic Geometry

$$d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

$$c^2 = a^2 + b^2$$

$$2 \cdot \text{Area} = |x_1(y_2-y_3) + x_2(y_3-y_1) + x_3(y_1-y_2)|$$

$$\text{Circle Area} = \pi r^2, \quad \text{Circumference} = 2\pi r$$

12.6 3D Formulas

$$\text{Cube:} \quad V = a^3, \quad A = 6a^2$$

$$\text{Cylinder:} \quad V = \pi r^2 h, \quad A = 2\pi r(r+h)$$

$$\text{Cone:} \quad V = \tfrac{1}{3}\pi r^2 h, \quad l = \sqrt{r^2+h^2}, \quad A = \pi r(r+l)$$

$$\text{Sphere:} \quad V = \tfrac{4}{3}\pi r^3, \quad A = 4\pi r^2$$

12.7 Series & Math Basics

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

$$t_n = a + (n-1)d, \qquad S_n = \frac{n}{2}[2a + (n-1)d]$$

$$\text{Percentage Change} = \frac{\text{New} - \text{Old}}{\text{Old}} \times 100\%$$

$$\left\lceil \frac{a}{b} \right\rceil = \left\lfloor \frac{a+b-1}{b} \right\rfloor$$

12.8 Permutation, Combination & Probability

$$P(n,r) = \frac{n!}{(n-r)!}, \qquad C(n,r) = \binom{n}{r} = \frac{n!}{r!\,(n-r)!}$$

$$\frac{A}{B} \bmod M = \left(A \times B^{M-2}\right) \bmod M$$

$$P(E) = \frac{\text{Favorable Outcomes}}{\text{Total Outcomes}}$$

$$\text{Ans} = \left(P \times Q^{M-2}\right) \bmod M$$

13 Sort

13.1 Custom Sort

```
bool cmp(Team a,Team b){
   if(a.solve!=b.solve)
      return a.solve>b.solve;
   return a.penalty<b.penalty;
}
```

14 Default Template

```
#include<bits/stdc++.h>
#define ll long long int
#define pb push_back
#define vc vector
#define vi vc<int>
#define vl vc<ll>
#define dbg(x) cerr<<x<<endl;
#define mp(x,y) make_pair(x,y)
#define yes cout<<"YES"<<endl;
#define no cout<<"NO"<<endl;
#define tst int t;cin>>t;while(t--)
#define srt(v) sort(v.begin(),v.end());
#define rsrt(v) sort(v.rbegin(),v.rend());
#define rj ios::sync_with_stdio(false);\
        cin.tie(0);
#define rvs(v) reverse(v.begin(),v.end());
#define F first
#define S second
#define MOD 1000000007
#define gcd(a,b) __gcd(a,b)
#define lcm(a,b) (a*b)/gcd(a,b)
#define PI 2*acos(0.0)
#define pii pair<int,int>
#define fr(i,a,b) for(ll i=a;i<=b;i++)
#define coutv(v) for(auto it:v)\
   cout<<it<<' ';cout<<endl;
#define cinv(v) for(auto &it:v)cin>>it;
#define all(v) v.begin(),v.end()
#define rall(v) v.rbegin(),v.rend()
#define ld long double
#define nl '\n'
const int N=1e5;
using namespace std;
mt19937 rng(chrono::steady_clock::
   now().time_since_epoch().count());
int my_rand(int l,int r){
   return uniform_int_distribution
      <int>(l,r)(rng);
}
void solve(){}
int main(){
   rj
```

```
    solve();
    return 0;
}
// Graph Moves:
// fx[]={+1,-1,+0,+0}
// fy[]={+0,+0,+1,-1}
// fx[]={0,0,1,-1,-1,1,-1,1}//King
// fy[]={-1,1,0,0,1,1,-1,-1}
// fx[]={-2,-2,-1,-1,1,1,2,2}//Knight
// fy[]={-1,1,-2,2,-2,2,-1,1}
```

15 Data Structures

15.1 Binary Indexed Tree

```
template <class DT>
struct BIT {
    int n;
    vector<DT> fanwick;
    BIT(){}
    BIT(int nn){
        n=nn;
        fanwick.assign(n+1,0);
    }
    DT presum(int i){
        DT an=0;
        for(;i>=1;i-=(i&-i))
            an+=fanwick[i];
        return an;
    }
    void update(int i,DT val){
        if(i<=0) return;
        for(;i<=n;i+=(i&-i))
            fanwick[i]+=val;
    }
    void update(int lf,int rt,DT val){
        update(lf,val);
        update(rt+1,-val);
    }
    DT query(int l,int r){
        return presum(r)-presum(l-1);
    }
};
```

15.2 Disjoint Set Union

```
struct DSU{
    vector<int>par,sz,rnk;
    DSU(int n){
        par.assign(n+1,0);
        rnk.assign(n+1,0);
        sz.assign(n+1,1);
        for(int i=1;i<=n;i++) par[i]=i;
    }
    int get_parent(int n){
        if(par[n]==n) return n;
        return par[n]=get_parent(par[n]);
    }
    void union_set(int x,int y){
        x=get_parent(x);
        y=get_parent(y);
        if(x!=y){
            if(rnk[x]>rnk[y]) swap(x,y);
            par[x]=y;
            if(rnk[x]==rnk[y]) rnk[y]++;
            sz[x]+=sz[y]; sz[y]=0;
        }
    }
    int get_size(int n){
        return sz[get_parent(n)];
    }
};
```

15.3 Segment Tree (Basic Max)

```
int v[N];
struct Stree{
#define lf (nd<<1)
#define rt ((nd<<1)|1)
    int t[4*N];
    static const int inf=-1e9;
    Stree(){memset(t,0,sizeof t);}
    void build(int nd,int lo,int hi){
        if(lo==hi){t[nd]=v[lo];return;}
        int mid=(lo+hi)>>1;
        build(lf,lo,mid);
        build(rt,mid+1,hi);
        t[nd]=max(t[lf],t[rt]);
    }
    void update(int nd,int lo,int hi,
                int i,int x){
        if(lo>i||hi<i) return;
        if(lo==hi&&lo==i){t[nd]=x;return;}
        int mid=(lo+hi)>>1;
        update(lf,lo,mid,i,x);
        update(rt,mid+1,hi,i,x);
        t[nd]=max(t[lf],t[rt]);
    }
    int query(int nd,int lo,int hi,
              int i,int j){
        if(lo>j||hi<i) return inf;
        if(lo>=i&&hi<=j) return t[nd];
        int mid=(lo+hi)>>1;
        return max(query(lf,lo,mid,i,j),
                   query(rt,mid+1,hi,i,j));
    }
};
```

15.4 Segment Tree with Lazy

```
struct Stree{
    long long t[4*N],lazy[4*N];
    Stree(){
        memset(t,0,sizeof t);
        memset(lazy,0,sizeof lazy);
    }
    inline void push(int nd,int lo,int hi){
        if(lazy[nd]==0) return;
        t[nd]+=lazy[nd]*(hi-lo+1);
        if(lo!=hi){
            lazy[lf]+=lazy[nd];
            lazy[rt]+=lazy[nd];
        }
        lazy[nd]=0;
    }
    inline void uthao(int nd){
        t[nd]=t[lf]+t[rt];
    }
    void build(int nd,int lo,int hi){
        lazy[nd]=0;
        if(lo==hi){t[nd]=v[lo];return;}
        int mid=(lo+hi)>>1;
        build(lf,lo,mid);
        build(rt,mid+1,hi);
        uthao(nd);
    }
```

```
   void update(int nd,int lo,int hi,
            int i,int j,ll val){
      push(nd,lo,hi);
      if(j<lo||hi<i) return;
      if(i<=lo&&hi<=j){
         lazy[nd]=val;
         push(nd,lo,hi); return;
      }
      int mid=(lo+hi)>>1;
      update(lf,lo,mid,i,j,val);
      update(rt,mid+1,hi,i,j,val);
      uthao(nd);
   }
   ll query(int nd,int lo,int hi,
          int i,int j){
      push(nd,lo,hi);
      if(i>hi||lo>j) return 0;
      if(i<=lo&&hi<=j) return t[nd];
      int mid=(lo+hi)>>1;
      return query(lf,lo,mid,i,j)
           +query(rt,mid+1,hi,i,j);
   }
};
```

15.5 Segment Tree (AP Lazy)

```
ll get_sum(ll a,ll n,ll d){
   return (((a*2)+(n-1)*d)*n)/2;
}
struct node{
   ll a,d;
   node operator+(const node &t)const{
      return{a+t.a,d+t.d};
   }
};
ll nth(ll a,ll n,ll d){return a+(n-1)*d;}
struct Stree{
   ll t[4*N]; node lazy[4*N];
   // push/update/query as usual
   // lazy[nd]={nth(a,lo-i+1,d),d}
};
```

15.6 Sparse Table (RMQ)

```
int v[N+5],sptab[N+5][26],Log[N+5];
void LogPreCalc(){
   Log[1]=0;
   for(int i=2;i<=N;i++)
      Log[i]=Log[i/2]+1;
}
void SPgen(int n){
   for(int i=1;i<=n;i++)
      sptab[i][0]=v[i];
   for(int j=1;(1<<j)<=n;j++)
      for(int i=1;i+(1<<j)-1<=n;i++)
         sptab[i][j]=min(
            sptab[i][j-1],
            sptab[i+(1<<(j-1))][j-1]);
}
int RmnQ(int lo,int hi){
   int x=Log[hi-lo+1];
   return min(sptab[lo][x],
            sptab[hi-(1<<x)+1][x]);
}
```

15.7 Monotonic Queue

```
struct monotonic_queue{
   deque<pair<int,int>>dq;
   void add(int val,int in){
      while(!dq.empty()&&
          dq.back().first>=val)
         dq.pop_back();
      dq.push_back({val,in});
   }
   void del(int in){
      if(!dq.empty()&&
        dq.front().second==in)
         dq.pop_front();
   }
};
```

15.8 Trie Tree

```
struct Trie{
   struct node{
      node* nxt[26];
      bool endmark;
      node(){
         for(int i=0;i<26;i++)
            nxt[i]=NULL;
         endmark=false;
      }
   }*root;
   Trie(){root=new node();}
   void Add(string &s){
      node* cur=root;
      for(int i=0;i<(int)s.size();i++){
         int c=s[i]-'a';
         if(cur->nxt[c]==NULL)
            cur->nxt[c]=new node();
         cur=cur->nxt[c];
      }
      cur->endmark=true;
   }
   int query(string &s){
      node* cur=root;
      for(int i=0;i<(int)s.size();i++){
         int c=s[i]-'a';
         if(cur->nxt[c]==NULL)
            return false;
         cur=cur->nxt[c];
      }
      return cur->endmark;
   }
};
```

15.9 Merge Sort Tree

```
class Merge_Sort_Tree{
public:
   vector<int> v;
   vector<vector<int>> tree;
   int n;
   Merge_Sort_Tree(int _n){
      n=_n; v.resize(n+1);
      tree.resize(4*n);
   }
   void build(int nd,int b,int e){
      if(b>e) return;
      if(b==e){
         tree[nd].push_back(v[b]);
         return;
      }
      int m=(b+e)/2;
      build(nd*2,b,m);
      build(nd*2+1,m+1,e);
      merge(tree[nd*2].begin(),
          tree[nd*2].end(),
```

```
            tree[nd*2+1].begin(),
            tree[nd*2+1].end(),
            back_inserter(tree[nd]));
    }
    int query(int nd,int b,int e,
            int l,int r,int k){
        if(b>r||e<l) return 0;
        if(l<=b&&e<=r){
            int pos=upper_bound(
                tree[nd].begin(),
                tree[nd].end(),k)
                -tree[nd].begin();
            return tree[nd].size()-pos;
        }
        int m=(b+e)/2;
        return query(nd*2,b,m,l,r,k)
             +query(nd*2+1,m+1,e,l,r,k);
    }
};
```

15.10 Mo's Algorithm

```
int sz;
struct query{
    int l,r,id;
    inline bool operator<(
        const query &t)const{
        return make_pair(l/sz,r)<
             make_pair(t.l/sz,t.r);
    }
}q[N];
ll v[N],fr_cnt[2*N],an=0,ans[N];
void add(int i){
    an+=fr_cnt[v[i]^k];
    fr_cnt[v[i]]++;
}
void del(int i){
    fr_cnt[v[i]]--;
    an-=fr_cnt[v[i]^k];
}
// In solve():
// sz=sqrt(n);
// sort(q,q+m);
// int l=1,r=0;
// expand/shrink with add/del
```

15.11 SQRT Decomposition

```
vector<long long> vcr;
vector<vector<long long>> blocks;
long long N,block_size;
void initialize(){
    block_size=sqrt(N);
    long long bn=-1;
    for(int i=0;i<N;++i){
        if(i%block_size==0){
            bn++;
            blocks.emplace_back();
        }
        blocks[bn].push_back(vcr[i]);
    }
    for(auto &b:blocks)
        sort(b.begin(),b.end());
}
```

15.12 Ordered Set (Pbds)

```
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template<typename T>
using ordered_set=tree<T,null_type,
    less<int>,rb_tree_tag,
    tree_order_statistics_node_update>;
// order_of_key(k) -> # items < k
// find_by_order(k)-> k-th element
template<typename T>
using ordered_multiset=tree<T,null_type,
    less_equal<T>,rb_tree_tag,
    tree_order_statistics_node_update>;
// For pair:
typedef tree<pair<int,int>,null_type,
    less<pair<int,int>>,rb_tree_tag,
    tree_order_statistics_node_update>
    ordered_set_pair;
struct custom_hash{
    size_t operator()(
        const pair<int,int>&x)const{
        return ((ll)x.first)^
             (((ll)x.second)<<32);
    }
};
```

16 Graph Theory

16.1 Bipartite Matching (Hopcroft-Karp)

```
vector<vector<int>> graph;
vector<int> match,dist;
bool bfs(){
    queue<int> Q;
    for(int i=1;i<lim;i++){
        if(match[i]==0){
            dist[i]=0; Q.push(i);
        } else dist[i]=INT_MAX;
    }
    dist[0]=INT_MAX;
    while(!Q.empty()){
        int u=Q.front(); Q.pop();
        if(u!=0)
            for(int v:graph[u])
                if(dist[match[v]]==INT_MAX){
                    dist[match[v]]=dist[u]+1;
                    Q.push(match[v]);
                }
    }
    return dist[0]!=INT_MAX;
}
bool dfs(int u){
    if(u!=0){
        for(int v:graph[u])
            if(dist[match[v]]==dist[u]+1)
                if(dfs(match[v])){
                    match[v]=u; match[u]=v;
                    return true;
                }
        dist[u]=INT_MAX; return false;
    }
    return true;
}
int hopcroft_karp(){
    int matching=0;
    while(bfs())
        for(int i=1;i<(int)match.size();i++)
            if(match[i]==0&&dfs(i))
                matching++;
```

```
    return matching;
}
```

16.2 SCC (Kosaraju)

```
class SCC{
public:
    vector<int> order; vector<bool> used;
    int n,rt; vi root;
    vector<vector<int>> adj,adj_rev;
    SCC(int _n):n(_n),rt(0){
        root.resize(n+1);
        used=vector<bool>(n+1);
        adj=adj_rev=
            vector<vector<int>>(n+1);
    }
    void dfs1(int nd){
        used[nd]=true;
        for(auto v:adj[nd])
            if(!used[v]) dfs1(v);
        order.push_back(nd);
    }
    void dfs2(int nd){
        root[nd]=rt; used[nd]=true;
        for(auto v:adj_rev[nd])
            if(!used[v]) dfs2(v);
    }
    void solve(){
        for(int i=1;i<=n;i++)
            if(!used[i]) dfs1(i);
        used=vector<bool>(n+1,false);
        reverse(all(order));
        for(auto i:order)
            if(!used[i]){++rt;dfs2(i);}
    }
};
```

16.3 MST (Kruskal)

```
// Requires DSU
struct edge{
    int u,v,w;
    bool operator<(const edge& cm)const{
        return w<cm.w;
    }
};
vector<edge> e;
ll mst(int n){
    sort(e.begin(),e.end());
    DSU d(n); ll cnt=0,s=0;
    for(auto &ei:e){
        int u=ei.u,v=ei.v;
        if(d.get_parent(u)!=
          d.get_parent(v)){
            d.union_set(u,v);
            cnt++; s+=(ll)ei.w;
            if(cnt==n-1) break;
        }
    }
    return s;
}
```

16.4 Topological Sort + Cycle

```
vector<int> v,adj[27];
int col[27]; bool isCycle=0;
void cycle_detection(int n){
    if(col[n]){
        if(col[n]==1) isCycle=1;
        return;
    }
    col[n]=1;
    for(auto it:adj[n]) cycle_detection(it);
    col[n]=2; v.push_back(n);
}
```

16.5 Articulation Bridge

```
class Find_bridge{
public:
    int n,Time,m;
    vector<vector<int>> adj;
    vector<bool> visit;
    vector<int> tim,low;
    vector<pair<int,int>> bridges;
    Find_bridge(int _n):n(_n){
        Time=0; adj.resize(n+1);
        visit.resize(n+1);
        tim.resize(n+1);
        low.resize(n+1,INT_MAX);
    }
    void dfs(int node,int par){
        tim[node]=low[node]=++Time;
        visit[node]=1;
        for(auto to:adj[node]){
            if(par==to) continue;
            if(visit[to])
                low[node]=min(
                    low[node],tim[to]);
            else{
                dfs(to,node);
                low[node]=min(
                    low[node],low[to]);
                if(low[to]>tim[node])
                    bridges.push_back(
                        {min(to,node),
                         max(to,node)});
            }
        }
    }
};
```

16.6 Articulation Point

```
class ArticulationPoint{
public:
    int n; vector<vector<int>> adj;
    vector<bool> visited,artPoint;
    vector<int> tin,low; int timer;
    ArticulationPoint(int _n):n(_n){
        timer=0;
        visited.assign(n+1,false);
        tin.assign(n+1,-1);
        low.assign(n+1,-1);
        artPoint.assign(n+1,false);
        adj.resize(n+1);
    }
    void dfs(int v,int p=-1){
        visited[v]=true;
        tin[v]=low[v]=timer++;
        int children=0;
        for(int to:adj[v]){
            if(to==p) continue;
            if(visited[to])
                low[v]=min(low[v],tin[to]);
            else{
                dfs(to,v);
                low[v]=min(low[v],low[to]);
                if(low[to]>=tin[v]&&p!=-1)
```

```
                artPoint[v]=true;
             ++children;
          }
       }
       if(p==-1&&children>1)
          artPoint[v]=true;
    }
    int find_cutpoints(){
       int cnt=0;
       for(int i=1;i<=n;++i){
          if(!visited[i]) dfs(i);
          cnt+=artPoint[i];
       }
       return cnt;
    }
};
```

16.7 LCA (Binary Lifting)

```
const int Log=15;
class Lca{
public:
   vector<vector<int>> up,g;
   vector<int> depth; int n;
   Lca(int _n):n(_n){
      up.resize(n+1,vector<int>(Log,-1));
      g.resize(n+1); depth.resize(n+1);
   }
   void dfs(int nd,int par,int d){
      depth[nd]=d; up[nd][0]=par;
      for(auto to:g[nd])
         if(to!=par) dfs(to,nd,d+1);
   }
   void build(){
      for(int i=1;i<Log;i++)
         for(int j=1;j<=n;j++)
            if(up[j][i-1]!=-1)
               up[j][i]=
                  up[up[j][i-1]][i-1];
   }
   int query(int u,int v){
      if(depth[u]<depth[v]) swap(u,v);
      ll diff=depth[u]-depth[v];
      for(ll i=0;i<Log;i++)
         if((diff>>i)&1) u=up[u][i];
      if(u==v) return u;
      for(ll i=Log-1;i>=0;i--)
         if(up[u][i]!=up[v][i]){
            u=up[u][i]; v=up[v][i];
         }
      return up[u][0];
   }
   int x_thParent(int u,int x){
      for(ll i=0;i<Log;i++)
         if((x>>i)&1) u=up[u][i];
      return u;
   }
   int dist(int a,int b,int l){
      return depth[a]+depth[b]-depth[l]*2;
   }
};
```

16.8 Centroid Decomposition

```
class CentroidDecomposition{
public:
   vector<vector<int>> adj,lightAdj;
   vector<bool> removed;
   vector<int> subtree,parent;
   int n,root;
   CentroidDecomposition(int _n):n(_n){
      root=1; adj.resize(n+1);
      lightAdj.resize(n+1);
      removed.resize(n+1);
      subtree.resize(n+1);
      parent.resize(n+1);
   }
   int get_sz(int node,int par=-1){
      subtree[node]=1;
      for(auto x:adj[node])
         if(!removed[x]&&par!=x)
            subtree[node]+=
               get_sz(x,node);
      return subtree[node];
   }
   int get_cen(int node,int des,
            int par=-1){
      for(auto x:adj[node])
         if(!removed[x]&&x!=par
           &&des<=subtree[x])
            return get_cen(x,des,node);
      return node;
   }
   void decompose(int node,int par=-1){
      int c=get_cen(node,get_sz(node)>>1);
      removed[c]=true;
      if(par!=-1){
         lightAdj[par].push_back(c);
         parent[c]=par;
      } else root=c;
      for(auto x:adj[c])
         if(!removed[x])
            decompose(x,c);
   }
};
```

16.9 Max Flow (Ford-Fulkerson)

```
struct Edge{int u,v,cap;};
vector<Edge> edge;
vector<int> edgeno[105];
bool vis[105]; int src,sink;
void AddEdge(int u,int v,int w){
   edge.push_back({u,v,w});
   edge.push_back({v,u,0});
   edgeno[u].push_back(edge.size()-2);
   edgeno[v].push_back(edge.size()-1);
}
int DFS(int u,int flow=1e9){
   vis[u]=1;
   if(u==sink) return flow;
   int an=0;
   for(int id:edgeno[u]){
      Edge &cr=edge[id];
      if(!cr.cap||vis[cr.v]) continue;
      an=DFS(cr.v,min(flow,cr.cap));
      if(an){
         edge[id^1].cap+=an;
         cr.cap-=an; return an;
      }
   }
   return an;
}
int FordFulkerson(){
   int MaxFlow=0;
```

```
    while(1){
        memset(vis,0,sizeof vis);
        int f=DFS(src);
        if(!f) break;
        MaxFlow+=f;
    }
    return MaxFlow;
}
```

16.10 Min Cost Max Flow

```
using T=long long; const T INF=1LL<<61;
struct MCMF{
    struct edge{int u,v,id;T cap,cost;};
    int n,s,t,mxid; T flow,cost;
    vector<vector<int>> g;
    vector<edge> e;
    vector<T> d,potential; vector<int> par;
    bool neg;
    MCMF(int _n):n(_n+10),neg(false),mxid(0)
        ↪{
        g.assign(n,{});
    }
    void add_edge(int u,int v,T cap,T cost,
              int id=-1,bool dir=true){
        if(cost<0) neg=true;
        g[u].push_back(e.size());
        e.push_back({u,v,id,cap,cost});
        g[v].push_back(e.size());
        e.push_back({v,u,-1,0,-cost});
        mxid=max(mxid,id);
        if(!dir) add_edge(v,u,cap,cost,-1);
    }
    bool dijkstra(){
        par.assign(n,-1); d.assign(n,INF);
        priority_queue<pair<T,T>,
            vector<pair<T,T>>,
            greater<pair<T,T>>> q;
        d[s]=0; q.push({0,s});
        while(!q.empty()){
            auto[nw,u]=q.top(); q.pop();
            if(nw!=d[u]) continue;
            for(int id:g[u]){
                int v=e[id].v;
                T cap=e[id].cap;
                T w=e[id].cost
                  +potential[u]-potential[v];
                if(d[u]+w<d[v]&&cap>0){
                    d[v]=d[u]+w;
                    par[v]=id; q.push({d[v],v});
                }
            }
        }
        for(int i=0;i<n;i++)
            if(d[i]<INF) potential[i]+=d[i];
        return d[t]!=INF;
    }
    T send(int v,T cur){
        if(par[v]==-1) return cur;
        int id=par[v]; int u=e[id].u;
        T f=send(u,min(cur,e[id].cap));
        cost+=f*e[id].cost;
        e[id].cap-=f; e[id^1].cap+=f;
        return f;
    }
    pair<T,T> solve(int _s,int _t,T goal=INF
        ↪){
        s=_s; t=_t; flow=0; cost=0;
        potential.assign(n,0);
        while(flow<goal&&dijkstra())
            flow+=send(t,goal-flow);
        return{flow,cost};
    }
};
```

16.11 Bellman-Ford / Neg. Cycle

```
// edges: {cost,{from,to}}
vector<pair<int,pair<int,int>>> edges;
vector<ll> dis(n+1,1LL<<62);
vector<int> parent(n+1);
// returns cycle nodes or empty
vi findNegCycle(int n){
    int last=-1; dis[1]=0;
    for(int i=0;i<=n;i++){
        last=-1;
        for(auto x:edges){
            int a=x.second.first;
            int b=x.second.second;
            ll c=x.first;
            if(dis[b]>dis[a]+c){
                dis[b]=dis[a]+c;
                last=b; parent[b]=a;
            }
        }
    }
    vi res; if(last==-1) return res;
    for(int i=0;i<n;i++)
        last=parent[last];
    for(int y=last;;y=parent[y]){
        res.push_back(y);
        if(y==last&&res.size()>1) break;
    }
    return res;
}
```

16.12 DSU on Tree

```
int sz[N],col[N],cnt_c[N];
vi nodes[N]; ll mxcol=0,sum[N],ans[N];
void count_size(int n,int p){
    sz[n]=1;
    for(auto it:adj[n]){
        if(it==p) continue;
        count_size(it,n); sz[n]+=sz[it];
    }
}
void dsu(int n,int p,bool keep){
    int Max=-1,big=-1;
    for(auto it:adj[n])
        if(it!=p&&sz[it]>Max){
            Max=sz[it]; big=it;
        }
    for(auto it:adj[n])
        if(it!=p&&it!=big) dsu(it,n,0);
    if(big!=-1){
        dsu(big,n,1);
        swap(nodes[n],nodes[big]);
    }
    nodes[n].push_back(n);
    cnt_c[col[n]]++;
    sum[cnt_c[col[n]]]+=col[n];
    if(cnt_c[col[n]]>mxcol) mxcol++;
    for(auto u:adj[n]){
        if(u==p||u==big) continue;
```

```
      for(auto it:nodes[u]){
         nodes[n].push_back(it);
         cnt_c[col[it]]++;
         sum[cnt_c[col[it]]]+=col[it];
         if(cnt_c[col[it]]>mxcol)
            mxcol++;
      }
   }
   ans[n]=sum[mxcol];
   if(!keep)
      for(auto it:nodes[n]){
         sum[cnt_c[col[it]]]-=col[it];
         cnt_c[col[it]]--;
         if(sum[mxcol]==0) mxcol--;
      }
}
```

17 Number Theory

17.1 Extended Euclid + Mod Inverse

```
ll extended_euclid(ll a,ll b,ll &x,ll &y){
   if(b==0){x=1;y=0;return a;}
   ll xx,yy;
   ll g=extended_euclid(b,a%b,xx,yy);
   x=yy; y=xx-yy*(a/b); return g;
}
ll mod_inverse(ll a,ll m){
   ll x,y;
   ll gc=extended_euclid(a,m,x,y);
   if(gc!=1) return -1;
   return (x%m+m)%m;
}
```

17.2 BigMod, Factorial, nCr

```
int BigMod(ll p,ll q,const int mod){
   int ans=1%mod; p%=mod;
   if(p<0) p+=mod;
   while(q){
      if(q&1) ans=(ll)ans*p%mod;
      p=(ll)p*p%mod; q>>=1;
   }
   return ans;
}
ll fact[N+5];
void pre(){
   fact[0]=1;
   fr(i,1,N){
      fact[i]=i*fact[i-1]%MOD;
   }
}
ll ncr(ll n,ll r){
   if(n<r) return 0;
   ll an=fact[n];
   ll d=fact[r]*fact[n-r]%MOD;
   an=an*BigMod(d,MOD-2,MOD)%MOD;
   return an;
}
```

17.3 Euler Totient (Sieve)

```
ll phi[N+5];
void euler(){
   for(ll i=1;i<=N;i++) phi[i]=i;
   for(ll i=2;i<=N;i++)
      if(phi[i]==i)
         for(ll j=i;j<=N;j+=i){
            phi[j]/=i;
            phi[j]*=(i-1);
         }
}
int Totient(int num){
   int ans=num;
   for(int i=2;i*i<=num;i++){
      if(num%i) continue;
      while(num%i==0) num/=i;
      ans-=ans/i;
   }
   if(num>1) ans-=ans/num;
   return ans;
}
```

17.4 Sieve of Eratosthenes

```
bool prime[N+5];
void primenire(){
   prime[0]=prime[1]=true;
   for(int i=4;i<=N;i+=2)
      prime[i]=true;
   for(int i=3;i*i<=N;i+=2)
      if(!prime[i])
         for(int j=i*i;j<=N;j+=2*i)
            prime[j]=true;
}
```

17.5 Miller-Rabin Primality

```
typedef unsigned long long ull;
ll bigmod_mr(ull n,ull p,ull m){
   ull x=1; n%=m;
   while(p){
      if(p&1) x=(__uint128_t)x*n%m;
      n=(__uint128_t)n*n%m; p>>=1;
   }
   return x;
}
bool isComposite(ull n,int p,ull d,ull m){
   ull x=bigmod_mr(n,d,m);
   if(x==1||x==m-1) return false;
   for(int i=1;i<p;i++){
      x=(__uint128_t)x*x%m;
      if(x==m-1) return false;
   }
   return true;
}
bool isPrime(ull n){
   if(n<2) return false;
   ull m=n; n--; int p=0;
   while(n%2==0) n>>=1,p++;
   for(auto i:{2,3,5,7,11,13,17,
               19,23,29,31,37}){
      if(m==(ull)i) return true;
      if(isComposite(i,p,n,m))
         return false;
   }
   return true;
}
```

17.6 Discrete Logarithm

```
// min x: a^x % m == b%m (gcd(a,m)=1)
int discrete_log(int a,int b,int m){
   a%=m; b%=m; int n=sqrt(m)+1;
   int an=1;
   for(int i=0;i<n;i++) an=an*1LL*a%m;
   unordered_map<int,int> vals;
   for(int q=0,cur=b;q<=n;q++){
      vals[cur]=q; cur=cur*1LL*a%m;
   }
```

```
    for(int p=1,cur=1;p<=n;p++){
        cur=cur*1LL*an%m;
        if(vals.count(cur))
            return n*p-vals[cur];
    }
    return -1;
}
```

17.7 Sum / Number of Divisors

```
// Requires prime[] from sieve
ll SumOfDivisor(ll n){
    ll ans=1;
    for(ll i=0;prime[i]*prime[i]<=n;i++){
        ll sum=0,p=1;
        while(n%prime[i]==0){
            n/=prime[i]; p*=prime[i];
            sum+=p;
        }
        ans*=(sum+1);
    }
    if(n>1) ans*=(n+1);
    return ans;
}
ll NumberOfDivisor(ll n){
    ll ans=1;
    for(ll i=0;prime[i]*prime[i]<=n;i++){
        ll cnt=0;
        while(n%prime[i]==0){
            n/=prime[i]; cnt++;
        }
        ans*=(cnt+1);
    }
    if(n>1) ans*=2;
    return ans;
}
```

18 String

18.1 KMP

```
template<typename T>
class kmp{
    vector<int> indx;
public:
    void lps(T &patt){
        indx.resize(patt.size(),0);
        int i=0,j=1;
        while(j<(int)patt.size()){
            if(patt[i]==patt[j])
                indx[j]=++i,j++;
            else{
                if(i!=0) i=indx[i-1];
                else indx[j]=0,j++;
            }
        }
    }
    bool match(T &text,T &patt){
        int i=0,j=0;
        while(j<(int)text.size()){
            if(patt[i]==text[j]) i++,j++;
            else{
                if(i!=0) i=indx[i-1];
                else j++;
            }
            if(i==(int)patt.size())
                return true;
        }
        return false;
    }
    int frequency(T &text,T &patt){
        int i=0,j=0,cnt=0;
        while(j<(int)text.size()){
            if(patt[i]==text[j]) i++,j++;
            else{
                if(i!=0) i=indx[i-1];
                else j++;
            }
            if(i==(int)patt.size())
                cnt++,i=indx[i-1];
        }
        return cnt;
    }
};
```

18.2 Z-Algorithm

```
vector<int> z_maker(string &s){
    int sz=s.size();
    vector<int> zv(sz); int l=0,r=0;
    for(int i=1;i<sz;i++){
        if(i<=r) zv[i]=min(zv[i-l],r-i+1);
        while(i+zv[i]<sz&&
              s[zv[i]]==s[i+zv[i]])
            zv[i]++;
        if(i+zv[i]-1>r) l=i,r=i+zv[i]-1;
    }
    return zv;
}
```

18.3 Hashing (Double Hash)

```
const int MOD1=127657753,MOD2=987654319;
const int b1=141,b2=277;
pair<int,int> pw[N],inpw[N];
int inp1,inp2;
void precalc(){
    pw[0]={1,1};
    for(int i=1;i<N;i++){
        pw[i].first=1LL*pw[i-1].first
                    *b1%MOD1;
        pw[i].second=1LL*pw[i-1].second
                     *b2%MOD2;
    }
    inp1=BigMod(b1,MOD1-2,MOD1);
    inp2=BigMod(b2,MOD2-2,MOD2);
    inpw[0]={1,1};
    for(int i=1;i<N;i++){
        inpw[i].first=1LL*inpw[i-1].first
                      *inp1%MOD1;
        inpw[i].second=1LL*inpw[i-1].second
                       *inp2%MOD2;
    }
}
struct Hashing{
    int sz; string s;
    vector<pair<int,int>> h_tab;
    Hashing(string _ss):sz(_ss.size()),
                        s(_ss){
        h_tab.emplace_back(0,0);
        for(int i=0;i<sz;i++){
            pair<int,int> p;
            p.first=(h_tab[i].first+
                1LL*pw[i].first*s[i]%MOD1)
                %MOD1;
            p.second=(h_tab[i].second+
```

```
            1LL*pw[i].second*s[i]%MOD2)
            %MOD2;
         h_tab.push_back(p);
      }
   }
   pair<int,int> get(int lo,int hi){
      pair<int,int> ans;
      ans.first=(h_tab[hi].first
         -h_tab[lo-1].first+MOD1)
         *1LL*inpw[lo-1].first%MOD1;
      ans.second=(h_tab[hi].second
         -h_tab[lo-1].second+MOD2)
         *1LL*inpw[lo-1].second%MOD2;
      return ans;
   }
};
```

18.4 Suffix Array

```
struct suf_array{
   string s; int sz;
   vector<int> st,rnk,lcp;
   suf_array(string &ss){
      s=ss; s+='$'; sz=s.size();
      st.resize(sz); rnk.resize(sz);
      lcp.resize(sz);
   }
   void count_sort(){
      vector<int> cnt(sz),pos(sz),sn(sz);
      for(auto x:rnk) cnt[x]++;
      pos[0]=0;
      for(int i=1;i<sz;i++)
         pos[i]=pos[i-1]+cnt[i-1];
      for(auto x:st){
         int i=rnk[x];
         sn[pos[i]]=x; pos[i]++;
      }
      st=sn;
   }
   void build(){
      vector<pair<char,int>> tm(sz);
      for(int i=0;i<sz;i++) tm[i]={s[i],i};
      sort(tm.begin(),tm.end());
      for(int i=0;i<sz;i++) st[i]=tm[i].
         ↪second;
      rnk[st[0]]=0;
      for(int i=1;i<sz;i++)
         rnk[st[i]]=(tm[i].first==
            tm[i-1].first)?
            rnk[st[i-1]]:rnk[st[i-1]]+1;
      int k=1;
      while(k<sz){
         for(int i=0;i<sz;i++)
            st[i]=(st[i]-k+sz)%sz;
         count_sort();
         vector<int> rn(sz); rn[st[0]]=0;
         for(int i=1;i<sz;i++){
            auto pre=make_pair(
               rnk[st[i-1]],
               rnk[(st[i-1]+k)%sz]);
            auto cr=make_pair(
               rnk[st[i]],
               rnk[(st[i]+k)%sz]);
            rn[st[i]]=(cr==pre)?
               rn[st[i-1]]:rn[st[i-1]]+1;
         }
         rnk=rn; k*=2;
      }
   }
};
```

18.5 Palindromic Tree (Eertree)

```
// Make string 1-indexed before use
struct Palindromic_tree{
   struct node{
      int nxt[26],sz,link,tot,fre,start;
   };
   string s; vector<node> t;
   int id,tt,a;
   Palindromic_tree(string ss):
      s(ss),a(ss.size()),t(ss.size()+3){
      t[1].sz=-1; t[1].link=1;
      t[2].sz=0; t[2].link=1;
      tt=id=2;
   }
   int Get_link(int x,int i){
      while(s[i-t[x].sz-1]!=s[i])
         x=t[x].link;
      return x;
   }
   bool Extend(int i){
      tt=Get_link(tt,i);
      int cur=Get_link(t[tt].link,i);
      int c=s[i]-'a';
      if(!t[tt].nxt[c]){
         t[tt].nxt[c]=++id;
         t[id].sz=t[tt].sz+2;
         t[id].link=(t[id].sz==1)?
            2:t[cur].nxt[c];
         t[id].tot=1+t[t[id].link].tot;
         tt=t[tt].nxt[c];
         t[tt].fre=1;
         t[tt].start=i-t[tt].sz+1;
         return true;
      }
      tt=t[tt].nxt[c]; t[tt].fre++;
      return false;
   }
};
```

18.6 Manacher's Algorithm

```
// d1[i]: radius odd palindrome at i
// d2[i]: radius even palindrome at i
vector<int> d1(n),d2(n);
for(int i=0,l=0,r=-1;i<n;i++){
   int k=(i>r)?1:min(d1[l+r-i],r-i);
   while(0<=i-k&&i+k<n&&
        s[i-k]==s[i+k]) k++;
   d1[i]=k--;
   if(i+k>r){l=i-k;r=i+k;}
}
for(int i=0,l=0,r=-1;i<n;i++){
   int k=(i>r)?0:min(d2[l+r-i+1],r-i+1);
   while(0<=i-k-1&&i+k<n&&
        s[i-k-1]==s[i+k]) k++;
   d2[i]=k--;
   if(i+k>r){l=i-k-1;r=i+k;}
}
```

18.7 String Matching (Bitset)

```
const int SZ=1e5+9;
bitset<SZ> bs[26],oc;
// Build: for(i=0;s[i];i++) bs[s[i]-'a'][i
   ↪]=1;
```

```
// Query pattern p:
// oc.set();
// for(i=0;p[i];i++) oc&=(bs[p[i]-'a']>>i);
// cout<<oc.count(); // occurrences
// int pos=oc._Find_first();
// Range [l,r]: (oc>>(l-1)).count()
//          -(oc>>(r-sz+1)).count()
```

19 Math & Matrix

19.1 Matrix Struct

```
struct Mat{
   vector<vector<int>> m; int x,y;
   Mat(int _x,int _y):x(_x),y(_y){
      m.assign(x,vector<int>(y,0));
   }
   inline Mat operator+(const Mat &mt){
      Mat ans(x,y);
      for(int i=0;i<x;i++)
         for(int j=0;j<y;j++)
            ans.m[i][j]=
               (m[i][j]+mt.m[i][j])%mod;
      return ans;
   }
   inline Mat operator*(const Mat &mt){
      Mat ans(x,mt.y);
      for(int i=0;i<x;i++)
         for(int j=0;j<mt.y;j++)
            for(int k=0;k<y;k++)
               ans.m[i][j]=(ans.m[i][j]+
                  1LL*m[i][k]*
                  mt.m[k][j]%mod)%mod;
      return ans;
   }
   inline void make_iden(){
      for(int i=0;i<x;i++)
         for(int j=0;j<x;j++)
            m[i][j]=(i==j);
   }
};
```

20 Geometry

```
typedef double T;
struct pt{
   T x,y;
   pt operator+(pt p){return{x+p.x,y+p.y};}
   pt operator-(pt p){return{x-p.x,y-p.y};}
   pt operator*(T d){return{x*d,y*d};}
   pt operator/(T d){return{x/d,y/d};}
};
T sq(pt p){return p.x*p.x+p.y*p.y;}
double absp(pt p){return sqrt(sq(p));}
T dot(pt v,pt w){return v.x*w.x+v.y*w.y;}
T cross(pt v,pt w){return v.x*w.y-v.y*w.x;}
T orient(pt a,pt b,pt c){
   return cross(b-a,c-a);
}
pt perp(pt p){return{-p.y,p.x};}
bool inDisk(pt a,pt b,pt p){
   return dot(a-p,b-p)<=0;
}
bool onSeg(pt a,pt b,pt p){
   return orient(a,b,p)==0&&inDisk(a,b,p);
}
double areaPoly(vector<pt> p){
   double area=0; int n=p.size();
   for(int i=0;i<n;i++)
      area+=cross(p[i],p[(i+1)%n]);
   return abs(area)/2.0;
}
bool inPolygon(vector<pt> p,pt a,
           bool strict=true){
   int cr=0; int n=p.size();
   for(int i=0;i<n;i++){
      if(onSeg(p[i],p[(i+1)%n],a))
         return !strict;
      cr+=(a.y>=p[i].y)!=
          (a.y>=p[(i+1)%n].y)&&
          orient(a,p[i],p[(i+1)%n])>0;
   }
   return cr&1;
}
struct line{
   pt v; T c;
   line(pt p,pt q):v(q-p),c(cross(v,p)){}
   T side(pt p){return cross(v,p)-c;}
   double dist(pt p){
      return abs(side(p))/absp(v);
   }
   bool inter(line l1,line l2,pt &out){
      T d=cross(l1.v,l2.v);
      if(d==0) return false;
      out=(l2.v*l1.c-l1.v*l2.c)/d;
      return true;
   }
};
```

20.1 Geometry Formulas

Triangle	
Circumradius	$R = \frac{abc}{4 \cdot \text{Area}}$
Inradius	$r = \frac{\text{Area}}{s}, s = \frac{a+b+c}{2}$
Heron	$\sqrt{s(s-a)(s-b)(s-c)}$
Cosine rule	$a^2 = b^2 + c^2 - 2bc\cos A$
Solids	
Cylinder	$V = \pi r^2 h$
Cone	$V = \frac{1}{3}\pi r^2 h$
Sphere	$A = 4\pi r^2, V = \frac{4}{3}\pi r^3$
Pick's	$A = I + \frac{B}{2} - 1$

21 Important Formulas

21.1 Useful Identities

- XOR of all subsets: $(\text{OR}) \cdot 2^{n-1}$
- Sum of all subarray sums: $(\text{sum}) \cdot 2^{n-1}$
- Derangement: $D_n = (n-1)(D_{n-1} + D_{n-2})$
- $\varphi(p^k) = p^k - p^{k-1}$
- $a^b \pmod{ac} \equiv a(b \bmod c)$
- Geometric sum: $\frac{a(1-r^n)}{1-r}$
- AP sum: $\frac{n(2a+(n-1)d)}{2}$

21.2 Bit Macros

```
__builtin_ffs(x) // least 1-bit pos
__builtin_clz(x) // leading zeros
__builtin_ctz(x) // trailing zeros
__builtin_popcount(x) // count 1-bits
// MSB: 32-__builtin_clz(x)
```